

Purpose:

This script automates the extraction of selected files from a ZIP archive and consolidates their contents into a single, human-readable text report. It targets Java source files (.java), Spring/Java configuration files (.properties), and the Maven project descriptor (pom.xml). The consolidated output is written to Combined_Contents.txt.

Typical Use Cases:

- Rapid code/content review after receiving a zipped project.
- Preparing a unified file for static analysis or text search across key source/config components.
- Sharing a compact snapshot of a project's important files with teammates or auditors.

Key Features

1. ZIP Extraction:

Unzips Example1.zip (Name can be any name) into an Extracted_App directory.

2. Selective File Aggregation:

Recursively walks the extracted directory and includes files matching:

- Extensions: .java, .properties
- Specific filenames: pom.xml

Each included file's content is appended to Combined_Contents.txt with a clear header indicating the source path.

3. Basic Error Handling:

If a file cannot be read (e.g., encoding issues), the script prints an error message but continues processing other files.

4. UTF-8 Safe Writing:

Writes the consolidated report using UTF-8 encoding to preserve non-ASCII characters in source or config files.

Prerequisites

- **Python:** 3.x with standard library modules zipfile and os (used for archive handling and directory traversal). The script also imports shutil but does not currently use it.

- **Input ZIP:** A file named Example1.zip (Name can be any name) placed in the same directory as py file.
- **Permissions:** Read access to the ZIP and write access to create Extracted_App and Combined_Contents.txt.

Here's why this is valuable in a business context:

1. Accelerates Code Review and Audits

Instead of manually opening multiple files across directories, the script gathers all critical files—Java classes, configuration (.properties), and Maven build descriptor (pom.xml)—into one document. This saves time for developers, auditors, and compliance teams.

2. Facilitates Knowledge Sharing

When onboarding new team members or collaborating with external vendors, providing a single consolidated file makes it easier to understand the project structure and dependencies without requiring full environment setup.

3. Supports Compliance and Governance

Organizations often need to review application configurations and dependencies for security or licensing compliance. This tool simplifies extracting relevant information for such checks.

4. Improves Documentation and Reporting

The output can serve as an artifact for internal documentation, project snapshots, or audit trails, ensuring transparency and traceability.

5. Enables Quick Impact Analysis

Before making changes or migrating systems, teams can quickly scan consolidated content to identify dependencies, configurations, and critical code segments.